Київський національний університет імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

ЗВІТ

з лабораторної роботи № 1 з дисципліни «Хмарні обчислення»

Тема 8.4: паралельне обчислення біноміальних коефіцієнтів, комбінацій і перестановок на платформі PARCS-Python у Google Cloud Platform

Виконав: студент групи ТТП-42 Бондаренко Максим

Київ — 2026

Мета роботи

Реалізувати паралельне обчислення C(n,k) і P(n,k) із застосуванням системи PARCS-Python, розгорнути кластер (master і чотири worker) у Google Cloud Platform, порівняти час послідовної та паралельної версій для 1, 2, 3 і 4 воркерів.

1. Ідея алгоритму

Кількість перестановок $P(n,k) = n!/(n-k)! = \prod(n-k+i)$, $i = 1..k$, а число комбінацій $C(n,k) = P(n,k)/k!$, де $k! = \prod i$. Усе зводиться до добутку чисел на діапазоні 1..k.

Master ділить діапазон 1..k на N відрізків, довжини яких відрізняються щонайбільше на 1 (map). Кожен воркер незалежно рахує добуток $(n-k+i)$ і добуток і на своєму відрізку. Master перемножує часткові добутки (reduce) і знаходить $C(n,k) = P(n,k)\cdot(k!)^{-1}$. Воркери не обмінюються даними між собою.

Для вимірювань використано режим mod (обчислення за модулем простого числа 1 000 000 007, потрібно n < модуль, ділення через обернений елемент). Вхід: n = 300 000 000, k = 150 000 000. Воркеру передається кілька чисел, а повертається пара чисел: обсяг мережевого обміну не залежить від розміру задачі, тому накладні витрати малі. Є також режим exact (точні великі числа, hex-рядки по мережі), але там master виконує послідовне множення й ділення великих чисел, і масштабування гірше, тому для замірів його не використано.

2. Помилки збірки та налаштування, пов'язані із застарілими версіями

1) Команда gcloud compute instances create-with-container, наведена в README parcs-python, більше не працює: Google припинила підтримку контейнерного агента. Замість неї створюються звичайні VM з Ubuntu 22.04, а startup-скрипт (cloud/startup.sh) встановлює Docker і запускає образ hummer12007/parcs-node з --network host. Роль (master або worker і IP master) передається в метаданих VM.

2) PARCS-Python працює на Python 2.7. У ньому range() створює список у пам'яті: на діапазоні 150 млн елементів процес воркера завершувався з «Killed» (нестача пам'яті). Виправлення: на початку модуля try: range = xrange / except NameError: pass. Код також залишається сумісним з Python 3.

3) Помилка в одній задачі зупиняє потік-планувальник master (виняток у destroy_workers), і нові задачі залишаються у стані Not Started. Лікування: перезапустити контейнер parcs на master (docker restart parcs), а потім на воркерах, щоб вони заново зареєструвались.

4) У зоні europe-north1-a виник stockout (немає вільних ресурсів); використано європейську зону europe-west4-a і тип e2-medium.

5) Із нового мережевого оточення порт 8080 master був недоступний напряму, тому веб-інтерфейс PARCS відкривався через SSH-тунель (gcloud compute ssh master -- -N -L 8081:localhost:8080).

3. Налаштування хмарного сервісу (команди)

```
# gcloud auth login; створити проєкт, підключити білінг, увімкнути API
gcloud projects create <PROJECT>
gcloud config set project <PROJECT>
gcloud billing projects link <PROJECT> --billing-account=<BILLING_ID>
gcloud services enable compute.googleapis.com
gcloud config set compute/zone europe-west4-a

# firewall: трафік усередині мережі + веб-інтерфейс лише з вашої IP
gcloud compute firewall-rules create parcs-internal --network=default --direction=INGRESS
--action=ALLOW --rules=all --source-ranges=10.0.0.0/8
gcloud compute firewall-rules create parcs-web --network=default --direction=INGRESS --
action=ALLOW --rules=tcp:8080,tcp:22 --source-ranges=<ВАША_IP>/32

# master
gcloud compute instances create master --machine-type=e2-medium --image-
family=ubuntu-2204-lts --image-project=ubuntu-os-cloud --metadata=parcs-args=master --
metadata-from-file=startup-script=cloud/startup.sh
# worker1..worker4 (IP master — внутрішня)
gcloud compute instances create worker1 worker2 worker3 worker4 --machine-type=e2-medium
--image-family=ubuntu-2204-lts --image-project=ubuntu-os-cloud --metadata="parcs-
args=worker <MASTER_INTERNAL_IP>" --metadata-from-file=startup-script=cloud/startup.sh

# запуск замірів (3 повтори для N = 1..4)
./cloud/run_all.sh <MASTER_IP:8080> 3
# після замірів — видалити VM
gcloud compute instances delete master worker1 worker2 worker3 worker4
```

Веб-інтерфейс PARCS: http://<MASTER_IP>:8080 (Add Job: solution.py та input.txt). Доступ відкрито лише з IP автора.

4. Кластер і виконання

```
$ gcloud compute instances list

NAME     ZONE            MACHINE_TYPE  INTERNAL_IP  EXTERNAL_IP     STATUS
master   europe-west4-a  e2-medium     10.164.0.2   34.187.119.248  RUNNING
worker1  europe-west4-a  e2-medium     10.164.0.5   34.34.104.165   RUNNING
worker2  europe-west4-a  e2-medium     10.164.0.4   34.6.39.50      RUNNING
worker3  europe-west4-a  e2-medium     10.164.0.6   34.147.14.206   RUNNING
worker4  europe-west4-a  e2-medium     10.164.0.3   35.234.166.36   RUNNING
```

Рис. 1. Віртуальні машини: master і worker1–worker4

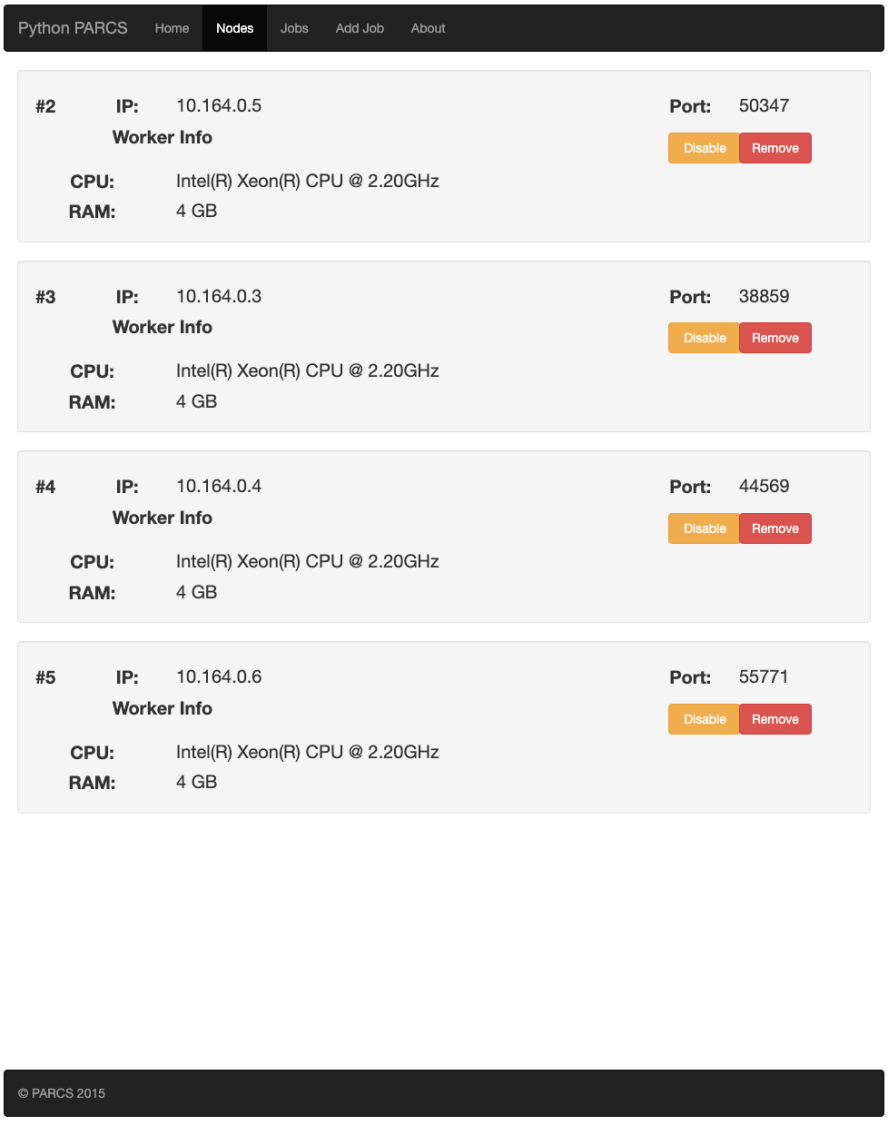


Рис. 2. Воркери, зареєстровані в PARCS (вкладка Nodes)

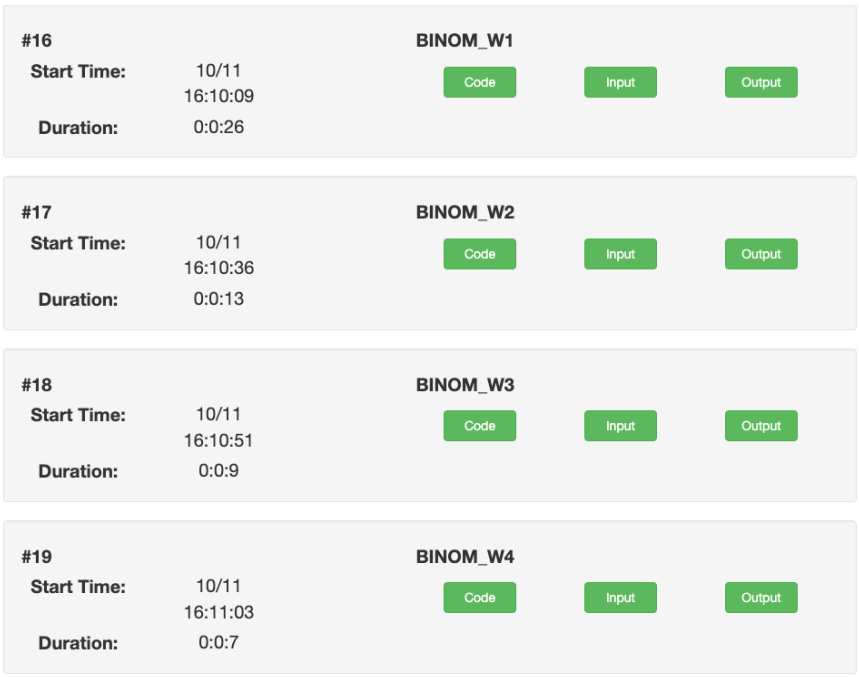


Рис. 3. Запуски для N = 1, 2, 3, 4 воркерів (вкладка Jobs)

5. Результати

Час виконання виміряно всередині задачі (solve), медіана з трьох прогонів. Послідовний еталон — sequential.py під Python 2 на тій самій конфігурації VM (медіана трьох). Усі прогони дали C(n,k) mod 1e9+7 = 940400276 і P(n,k) mod 1e9+7 = 591392064, що збігається з послідовною версією й з math.comb/math.perm на малих вхідних даних.

Воркерів	Час, с	Прискорення (відносно 1 воркера)
послідовна	26.87	—
1	24.75	1.00×
2	12.36	2.00×
3	8.36	2.96×
4	6.33	3.91×

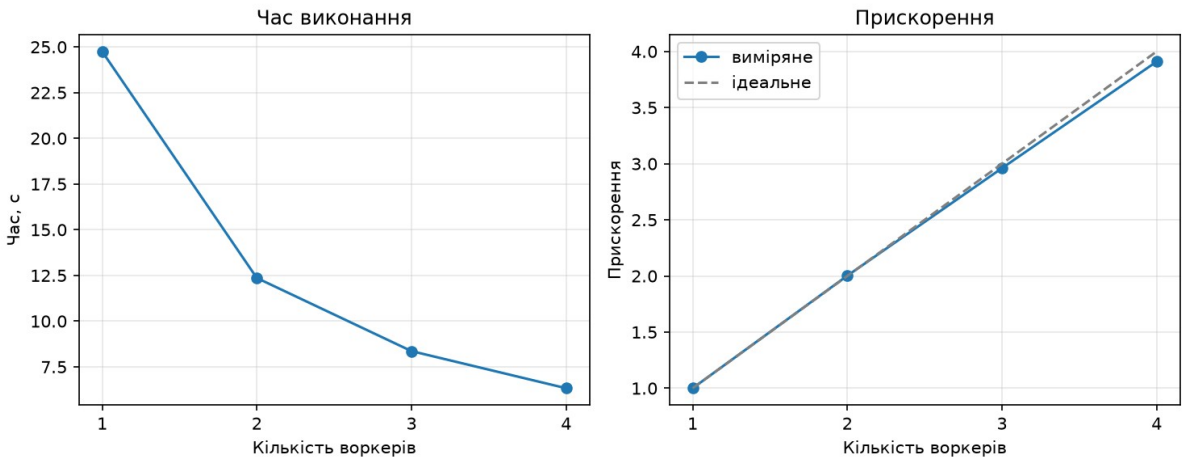


Рис. 4. Залежність часу виконання й прискорення від кількості воркерів

Час майже обернено пропорційний кількості воркерів: прискорення 2,00× для двох, 2,96× для трьох і 3,91× для чотирьох воркерів, що близько до ідеального. Невеликий відрив від ідеалу пояснюється фіксованими витратами на запуск задачі та розсилку. Час з одним воркером (24,75 с) трохи менший за послідовний (26,87 с), але в межах розкиду замірів на спільній VM (послідовні прогони дали 26,4–36,2 с).

Висновки

Задача біноміальних коефіцієнтів добре розпаралелюється на PARCS: обчислень багато, а даних для передачі мало, тому майже лінійне прискорення зберігається до чотирьох воркерів. Основні труднощі були пов’язані із застарілістю платформи: Python 2 і непрацюючий спосіб розгортання контейнерів у GCP.

Посилання на GitHub

https://github.com/dissimilatee/Lab1XO